

C++ Cheat sheet

Defining Variables

In C++, I define a variable by writing its type followed by its name.

C++

```
int age;  
double price = 12.5;  
bool isReady = true;  
char grade = 'A';  
string name = "Alex";
```

Common Variable Type

Type	Purpose	Example
int	Whole numbers	int age = 20;
double	Decimal numbers	double price = 3.14;
float	Decimal numbers with less precision	float value = 2.5f;
bool	True or false	bool passes = true;
char	One character	char grade = 'A';
string	Text	string name = "Sam";

To use string:

C++

```
#include <string>
```

Arrays

An array stores multiple values of the same type.

```
C++  
int numbers[5];
```

Assigning values:

```
C++  
numbers[0] = 10;  
numbers[1] = 20;
```

Reading a value:

```
C++  
cout << numbers[0];
```

Array indexes begin at 0, so an array of size 5 has indexes 0 to 4.

An array can also be initialized directly:

```
C++  
int numbers[5] = {10, 20, 30, 40, 50};
```

2D Arrays

A two-dimensional array stores values in rows and columns.

```
C++  
  
int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Accessing a value:

```
C++  
  
cout << matrix[1][2];
```

This prints 6.

Input and Output

Use cout to print output:

```
C++  
  
cout << "Hello" << endl;
```

Use cin to read input:

```
C++  
  
int age;  
cin >> age;
```

Multiple values could be read together:

C++

```
int a;  
int b;  
  
cin >> a >> b;
```

Conditions and If statements

An if statement runs code when a condition is true.

C++

```
if (age >= 18) {  
    cout << "Adult" << endl;  
}
```

Using else:

C++

```
if (score >= 50) {  
    cout << "Pass" << endl;  
} else {  
    cout << "Fail" << endl;  
}
```

Using else if:

C++

```
if (score >= 80) {  
    cout << "High Distinction" << endl;  
} else if (score >= 70) {  
    cout << "Distinction" << endl;  
} else {  
    cout << "Other grade" << endl;  
}
```

Common comparison operations:

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Logical operators:

Operator	Meaning
&&	AND
	OR
!	NOT

For loops:

A for loop is useful when the number of repetitions is known.

```
C++  
  
for (int i = 0; i < 5; i++) {  
    cout << i << endl;  
}
```

Reading values into an array:

```
C++  
  
for (int i = 0; i < n; i++) {  
    cin >> numbers[i];  
}
```

While Loops

A while loop continues while its condition is true.

```
C++  
  
int i = 0;  
  
while (i < 5) {  
    cout << i << endl;  
    i++;  
}
```